# Can LLMs Implement Architectural Tactics? Early Results

Anonymous Author(s)

## Abstract

While LLMs have shown strong code refactoring capabilities, their potential for architecture-level quality improvement remains unexplored. We present an automated pipeline that detects repository architecture, selects an appropriate tactic from a predefined catalog, and implements it through iterative code modifications. Applied to 162 open-source Python repositories, the pipeline completed 121 cases: 22 showed MI improvement (mean $\Delta$MI = +2.89 among improved), 92 remained stable, and 7 degraded. A Wilcoxon signed-rank test confirmed statistical significance ($W = 360.0$, $p = 0.001$, $r = 0.28$). Script-based repositories with Decomposability showed the strongest gains. This work provides early empirical evidence on both the potential and limitations of LLMs for architecture-level modifications.

## CCS Concepts

• **Software and its engineering** → **Software maintenance tools**; *Software architectures*; • **Computing methodologies** → *Natural language processing*.

## Keywords

software maintainability, architectural tactics, large language models, automated software improvement, static analysis, software architecture

**ACM Reference Format:**

Anonymous Author(s). 2026. Can LLMs Implement Architectural Tactics? Early Results. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Modern software engineering faces a persistent maintenance challenge, with studies indicating that 60–80% of total lifecycle costs are dedicated to post-deployment modifications [2]. While code-level refactoring is well-established [6], and industrial studies confirm that developers regularly refactor to improve maintainability despite significant effort and risk [10], there is a significant gap in the automated application of higher-level design decisions. We define this as the *Transformation Gap*—the disconnect between detecting architectural deficiencies through static analysis or expert review and automatically implementing design-level corrections that address them. While recent work has shown that LLMs can perform code-level refactoring effectively [4, 20], no existing approach reasons about the repository's architectural style to select and implement design-level tactics that systematically target quality attributes. This paper investigates the intersection of architectural tactics, quality attribute models, and the emerging capabilities of Large Language Models (LLMs) in automating structural transformations.

We address the following research questions:

- **RQ1:** What architectural classifications and tactic selections does the LLM-based pipeline produce, and how coherent are they with repository structure?
- **RQ2:** To what extent do LLM-implemented architectural tactics improve software maintainability as measured by the Maintainability Index?
- **RQ3:** How do architecture type and tactic choice influence improvement outcomes?

The remainder of this paper is organized as follows: Section 2 reviews background and related work; Section 3 describes the methodology; Section 4 details the implementation; Section 5 presents results and discussion; Section 6 discusses threats to validity; and Section 7 concludes.

## 2 Background and Related Work

### 2.1 Architectural Tactics and Quality Attributes

Software architecture is fundamentally defined by its elements, form, and rationale [18], or alternatively, as a collection of components and connectors in a specific configuration [7]. Within these frameworks, *architectural tactics* serve as fine-grained design decisions intended to achieve specific quality attribute responses [2]. Unlike architectural patterns (e.g., MVC, Layered), which provide broad structural templates, tactics target specific quality concerns at a finer granularity—for instance, "Reduce Coupling" targets modifiability, while "Decomposability" targets analysability and modularity. Kim et al. demonstrated that quality-driven architecture development using tactics can systematically improve targeted quality attributes when tactics are selected based on measured deficiencies [11]. Bogner et al. further showed that modifiability tactics—particularly coupling reduction—are systematically addressed through service-oriented design patterns in both SOA and microservice architectures [3].

The ISO/IEC 25010 standard [1] decomposes maintainability into five sub-characteristics: modularity, analysability, modifiability, reusability, and testability. The Maintainability Index (MI)—a composite metric combining Halstead Volume [8], Cyclomatic Complexity [16], and lines of code—has been widely adopted as a practical proxy for these sub-characteristics [17]. However, its validity remains debated: Lenarduzzi et al. found less than 0.4% agreement among six static analysis tools on quality issues [12], highlighting the need for multi-metric evaluation.



*EASE 2026, Glasgow, United Kingdom*



ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Research by Rahmati suggests that while architectural patterns provide a structural foundation, their effectiveness is highly context-dependent; misapplication can lead to a 10% increase in maintenance effort [21]. Márquez et al. conducted a systematic mapping of architectural tactics across 12 quality attributes based on 91 primary studies [14]. Their catalog, which maps each tactic to its primary quality attribute impact (e.g., Decomposability → Analysability, Reduced Coupling → Modifiability), forms the basis for our automated tactic selection mechanism.

## 2.2 LLMs in Software Refactoring

Recent multi-agent frameworks such as MANTRA have demonstrated an 82.8% success rate in producing compilable, test-passing refactored code through contextual RAG and multi-agent LLM collaboration [23]. However, empirical analysis of AI coding agents reveals a significant scale limitation: agentic refactorings are dominated by low-level, localized edits (renaming, type changes), with agents performing fewer high-level design changes than human developers [9]. This localized scope is particularly problematic for architectural tactics, which by nature require coordinated changes across multiple modules.

Research into prompt engineering indicates that while LLMs can identify up to 86.7% of refactoring opportunities, their performance in complex tactic synthesis remains limited [13]. Piao et al. explored bridging human expertise with LLM machine understanding for refactoring, finding that structured prompts with architectural context significantly improve output quality [19]. In security-related tactic implementation, Shokri and Khatchadourian found that models may achieve high syntactic correctness (95%) but fail significantly in semantic correctness (5%) due to an inability to coordinate inter-procedural changes across multiple classes [22]. Martinez et al. conducted a systematic literature review of software refactoring with LLMs, confirming that architecture-level transformations remain an open challenge [15]. Esposito et al. surveyed generative AI applications for software architecture, identifying maintainability improvement as a key open area [5].

Recent work has begun applying LLMs directly to maintainability improvement. Changizi et al. showed that LLM-assisted refactoring can improve MI at the file level [4], and Pucho et al. demonstrated a multi-agent LLM system for refactoring Python ML projects [20]. However, both approaches apply refactoring patterns directly (e.g., Extract Method, Rename) without reasoning about the repository's *architectural style* to select *design-level tactics* that systematically target quality attributes. Our pipeline introduces this intermediate architecture-aware reasoning layer.

## 2.3 Research Gaps

The synthesis of current literature identifies three key gaps that motivate this work:

- **Contextual Constraints:** Current LLM-based tools operate primarily at the method or class level, lacking system-wide awareness for architectural restructuring [9, 15]. No existing approach provides the LLM with a full repository-level architectural context.
- **Measurement Inconsistency:** There is a documented lack of agreement among static analysis tools (less than 0.4% in some cases), complicating quality validation [12]. This makes it difficult to reliably assess whether LLM-driven changes genuinely improve quality.
- **Implementation Void:** While methods exist for *detecting* the need for architectural tactics [14] and for *recovering* architectural decisions [22], no automated pipeline exists that closes the loop from detection through selection to concrete, multi-file implementation. A systematic literature review by Martinez et al. confirms that architecture-level refactoring remains an open challenge, with the vast majority of LLM-based studies addressing only code-level transformations [15].

# 3 Methodology

## 3.1 Research Design

This study employs a quantitative pre-test/post-test experimental design following the principles of evidence-based software engineering [2]. The independent variable is the application of LLM-based architectural tactics; the dependent variable is software maintainability as measured by a multi-metric suite. Each repository serves as its own control, allowing paired-sample comparison before and after the intervention, thereby mitigating the impact of inherent cross-project variance in code style, domain, and complexity.

The null hypothesis is that the application of LLM-implemented architectural tactics does not produce a statistically significant change in the Maintainability Index. To test this, we use a Wilcoxon signed-rank test (appropriate for non-normally distributed paired samples) and report the matched-pairs rank-biserial correlation as a non-parametric effect size measure.

## 3.2 Experimental Pipeline

The pipeline is structured into seven sequential phases, following a Pipes and Filters architecture where each phase operates as an independent filter consuming and producing structured JSON artifacts:

(1) **Repository selection:** Automated search and filtering of GitHub repositories matching predefined criteria;
(2) **Baseline static analysis:** Computation of MI, fan-out, docstring coverage, and package structure for the unmodified codebase;
(3) **Architecture detection:** LLM-based inference of the dominant architectural pattern from the repository's structure and import graph;
(4) **Tactic selection:** LLM-based selection of an appropriate architectural tactic from a predefined catalog, based on the detected architecture and identified structural deficiencies;
(5) **Tactic implementation:** LLM-generated code modifications applying the selected tactic through iterative incremental changes;
(6) **Post-intervention static analysis:** Recomputation of the same metric suite on the modified codebase;
(7) **Statistical analysis:** Paired comparison of before/after metrics with hypothesis testing.

Each phase is fully automated and operates without manual intervention. Repositories that fail at any phase (e.g., due to syntax

errors introduced during implementation) are recorded as "null" outcomes but retained in the dataset for failure rate analysis.

### 3.3 Dataset Preparation

Repositories were sourced from public Python-based backend projects on GitHub via the GitHub Search API. A multi-stage filtering process excluded toy projects and focused on maintainable industrial-like codebases [21]. The inclusion criteria required:

- Presence of a valid `requirements.txt` for dependency analysis;
- Presence of Python source files (`.py`) to ensure the project contains analyzable code;
- Exclusion of ML-heavy projects identified by the presence of Jupyter Notebooks (`.ipynb`) or dominant use of data science libraries;
- Assessment of project maturity through metadata (star counts across multiple ranges from 20 to 12,000, commit history indicating active development).

After filtering, 162 repositories were retained for the experiment, spanning domains including web frameworks, CLI tools, API clients, and automation utilities.

### 3.4 Maintainability Measurement

The evaluation framework is aligned with the ISO/IEC 25010 quality model [1], which decomposes maintainability into modularity, analysability, modifiability, reusability, and testability. The primary quantitative metric is the Maintainability Index (MI), calculated via the Radon static analysis framework [17]. MI is computed as a weighted composite of Halstead Volume [8], Cyclomatic Complexity [16], and lines of code, yielding a score from 0 (unmaintainable) to 100 (highly maintainable).

Recognizing the documented inconsistency among static analysis tools—where Lenarduzzi et al. found less than 0.4% agreement among six tools [12]—we supplement MI with a multi-dimensional metric suite to triangulate maintainability changes:

- **Structural Complexity:** Average fan-out (mean imports per module) and package count, serving as proxies for coupling and modularity;
- **Documentation Quality:** Docstring coverage ratio (proportion of functions/classes with docstrings);
- **Delta Measurement:** $\Delta MI = MI_{after} - MI_{before}$, providing a normalized measure of quality change per repository.

### 3.5 Tactic Catalog

The tactic selection is based on the catalog established by Bass et al. [2] and systematically mapped by Márquez et al. [14]. We focus on four maintainability-related tactics applicable to Python codebases:

- **Decomposability:** Splitting monolithic modules into smaller, cohesive units to improve analysability and modularity;
- **Reduced Coupling:** Introducing interfaces or mediators to minimize inter-module dependencies, targeting modifiability;
- **Localized Modification:** Encapsulating change-prone logic to reduce the ripple effect of future modifications;
- **Deferred Binding Time:** Externalizing configuration or using dependency injection to improve flexibility.

The LLM selects the most appropriate tactic based on the detected architecture and the repository's structural characteristics, providing a justification for its choice.

### 3.6 LLM-Driven Transformation

The transformation operates in three sequential layers: (1) **Architecture Detection** classifies the repository into an architectural style with supporting evidence; (2) **Tactic Selection** identifies structural deficiencies and selects an appropriate tactic from the catalog; (3) **Tactic Implementation** applies incremental code modifications, each validated for syntactic correctness. While literature suggests that Test-Driven Development (TDD) significantly improves reliability of LLM refactoring [19, 23], a full TDD cycle is designated for **future work**. Section 4 details the technical realization of each layer.

## 4 Implementation

The framework is implemented as an automated Python-based orchestration engine using a Pipes and Filters architecture. Each filter in the pipeline encapsulates a single responsibility (e.g., static analysis, architecture detection) and communicates via structured JSON artifacts, enabling modular extension and reproducibility. Target repositories are shallow-cloned, processed sequentially, and purged after data extraction to ensure isolation.

The LLM component uses Qwen3-coder-next:cloud (via Ollama, 256k context window, temperature = 0.2). We selected this model for its 256k token context window—necessary for repository-level analysis where file trees, import graphs, and multi-file code must be provided in a single prompt—and its support for local deployment via Ollama, ensuring reproducibility without API rate limits or cost constraints. Comparing across multiple LLMs (e.g., GPT-4, Claude, Gemini) is left for future work. Each prompt uses structured engineering enforcing JSON-formatted outputs and includes (a) the repository's file tree, (b) summarized imports and function signatures per file, (c) baseline static analysis metrics, and (d) task-specific instructions requiring structured JSON responses. This global architectural context—provided through the 256k context window—enables the LLM to reason about cross-module dependencies and coordinate changes across multiple files, directly addressing the single-file limitation identified in recent studies [19].

For architecture detection, the LLM classifies the repository into one of several architectural styles (e.g., modular monolith, layered, script-based) with a confidence score and supporting evidence. For tactic selection, it chooses from a catalog of 32 architectural tactics [2, 14], providing justification and expected impact. For implementation, the LLM generates concrete code modifications following an iterative incremental strategy—applying small, self-contained changes and verifying that the modified code remains syntactically valid.

All artifacts, including LLM reasoning logs, intermediate code states, and metric snapshots, are preserved for full traceability of the architectural evolution.

# 5 Results and Discussion

## 5.1 Overall Quantitative Impact

Table 1 summarizes the overall results across 162 repositories. Of these, 41 (25.3%) failed to complete the pipeline (null `mi_after`), leaving 121 with paired before/after measurements.

**Table 1: Descriptive statistics for repositories with paired MI measurements (N = 121).**

| Metric | $MI_{before}$ | $MI_{after}$ |
|---|---|---|
| Mean | 65.11 | 65.59 |
| Median | 68.42 | 68.67 |
| Std. Dev. | 19.17 | 19.22 |
| Mean ΔMI | +0.48 | |
| Median ΔMI | 0.00 | |

A Shapiro-Wilk test confirmed non-normality of the delta distribution ($W = 0.34$, $p < 0.001$), justifying the use of a non-parametric test. A Wilcoxon signed-rank test on all 121 paired observations ($W = 360.0$, $p = 0.001$, 29 non-zero differences) indicates a statistically significant positive shift in MI. The matched-pairs rank-biserial correlation is $r = 0.28$, indicating a small effect size, reflecting that the magnitude of improvement is modest in aggregate.

As illustrated in Figure 1, a majority of projects maintained baseline stability. In 76.0% of completed cases (92 of 121), the MI remained unchanged, suggesting the LLM's internal refactoring did not always produce changes detectable by Radon's MI formula.

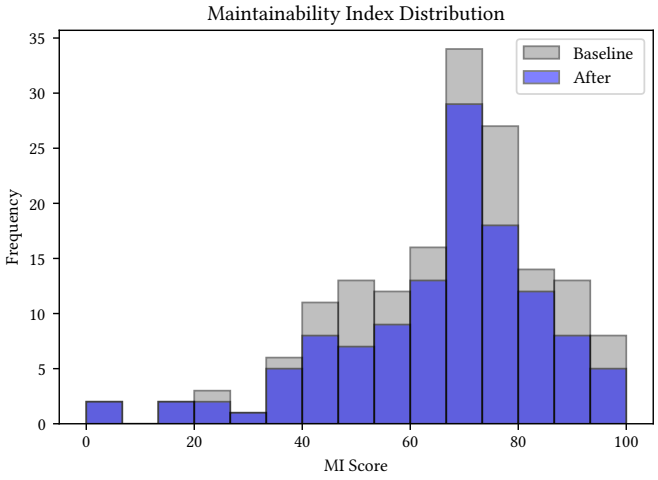


**Figure 1: Distribution of MI scores: Baseline vs. Post-Intervention.**

## 5.2 Success Cases and Tactic Efficacy

22 repositories (13.6%) showed measurable MI improvement (Figure 2). The largest gain was in *get_subscribe* (ΔMI = +13.52), where the **Decomposability** tactic decomposed large scripts into modular components—a change highly rewarded by the MI formula. Similarly, *whoogle-search* improved from 67.06 to 69.74 via **Localized Modification** in a layered architecture.

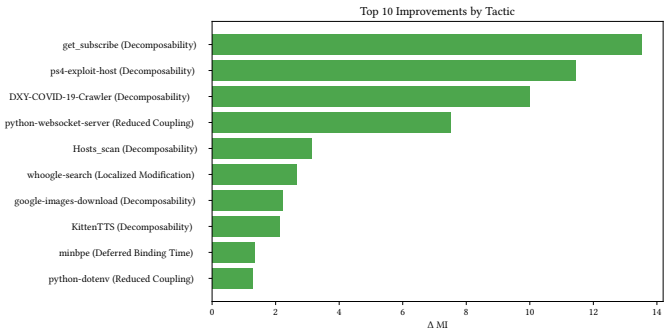


**Figure 2: Top 10 MI improvements by applied tactic.**

Table 2 breaks down results by tactic. **Reduced Coupling** showed the highest improvement rate (8/18 improved, 44.4%), while **Decomposability**—the most frequently selected tactic (103 repos, 63.6%)—had the highest null rate (29/103, 28.2%), suggesting that decomposing unstructured code is a higher-risk transformation. The low tactic diversity (only 4 of 32 catalog tactics selected) likely reflects the dataset composition: Python backend projects predominantly exhibit monolithic structure or loose coupling, making Decomposability the natural first-choice tactic.

**Table 2: Results by architectural tactic.**

| Tactic | N | Null | Impr. | Stable | Degr. | $\overline{\Delta MI}$ |
|---|---|---|---|---|---|---|
| Decomposability | 103 | 29 | 7 | 63 | 4 | +0.55 |
| Localized Mod. | 32 | 5 | 6 | 19 | 2 | +0.13 |
| Reduced Coupling | 23 | 5 | 8 | 9 | 1 | +0.70 |
| Other | 4 | 2 | 1 | 1 | 0 | +0.67 |

## 5.3 Failure Rates and Robustness

Of the 162 repositories, 41 (25.3%) resulted in pipeline failure (null status), representing cases where the LLM introduced syntax errors, broken dependencies, or changes that prevented execution. An additional 4.3% (7 repos) showed minor MI degradation (mean ΔMI = -0.78). This underscores the necessity of verification mechanisms for automated architectural modifications.

## 5.4 Architectural Influence

The dataset is dominated by **modular monolith** (51.2%) and **script-based** (43.8%) architectures, with layered (3.7%) and MVC (1.2%) as a small minority. Table 3 presents a per-architecture breakdown. Script-based projects showed the highest mean improvement (+0.81) and the three largest individual gains (get_subscribe +13.52, ps4-exploit-host +11.45, DXY-COVID-19-Crawler +9.99), all achieved via the Decomposability tactic. This is intuitive: script-based repositories often consist of monolithic files with high Halstead Volume, which are directly reduced by decomposition—a change heavily

rewarded by the MI formula. However, script-based projects also exhibited the highest pipeline failure rate (29.6%), reflecting the difficulty of restructuring code that lacks modular boundaries.

Modular monoliths showed more modest but consistent gains (+0.22), with 15 of 67 completed repositories improving. Notably, all 7 degradation cases occurred in modular monolith repositories. This asymmetry suggests that well-structured codebases are more sensitive to modifications: the LLM's changes may inadvertently increase coupling or complexity in already-organized code. The layered architecture, while represented by only 3 completed repositories, showed the highest mean improvement (+0.94) with zero degradations, hinting that layered structures may provide a favorable context for LLM-driven transformations, though this observation is based on only 3 completed repositories and cannot support statistical conclusions; it requires validation with a substantially larger sample.

**Table 3: Results by architecture type.**

| Architecture | N | Null% | Impr. | Degr. | $\overline{\Delta}$MI |
|---|---|---|---|---|---|
| Script-based | 71 | 29.6 | 5 | 0 | +0.81 |
| Modular mono. | 83 | 19.3 | 15 | 7 | +0.22 |
| Layered | 6 | 50.0 | 2 | 0 | +0.94 |
| MVC | 2 | 50.0 | 0 | 0 | 0.00 |

## 5.5 Discussion

Our results can be contextualized against prior findings. The 25.3% pipeline failure rate is notably lower than the 95% semantic failure rate reported by Shokri and Khatchadourian for security tactic implementation [22], though their task (inter-procedural synthesis) is arguably more complex. Our overall improvement rate (13.6%) is modest compared to MANTRA's 82.8% compilable refactoring rate [23]; however, MANTRA targets code-level refactoring within single files, while our pipeline attempts architecture-level changes across entire repositories.

The prevalence of stable results (76.0% of completed cases) raises an important methodological question: does unchanged MI imply no meaningful modification? Among the 53 repositories with preserved tactic application artifacts, all contained at least one code modification step (mean 4.7 steps per repository), confirming that the LLM did produce code changes even in MI-stable cases. These modifications (e.g., extracting helper classes, reorganizing imports) may improve readability or separation of concerns without affecting MI's complexity-based formula. This aligns with the known limitation of MI as a quality proxy [12] and suggests that complementary evaluation methods (e.g., expert review, coupling metrics) are needed to fully assess the impact of architectural improvements.

While the aggregate effect ($\Delta$MI = +0.48) is small, the improved subset shows a practically meaningful mean gain of +2.89, with individual improvements exceeding +10 MI points. This suggests that the pipeline's value lies not in uniform improvement but in identifying and transforming receptive repositories. The strong performance of the Decomposability tactic on script-based repositories points to a practical sweet spot: projects with low initial structure and high file-level complexity offer the clearest path for automated decomposition, where even simple modularization yields measurable MI gains. This finding echoes Kim et al.'s industrial study at Microsoft [10], which identified that developers most frequently refactor to improve readability and decomposition—precisely the transformations where our pipeline showed the greatest success.

## 5.6 Supplementary Metric Analysis

To assess whether LLM-driven tactics affected dimensions beyond MI, we examined structural and documentation metrics across the 123 repositories with paired before/after static analysis artifacts. Given the small number of repositories with non-zero changes (28/123 for fan-out, 17/123 for docstring coverage), formal hypothesis testing was not meaningful for these metrics; we report descriptive statistics only.

**Fan-out.** Average fan-out (mean number of imports per module) changed in 28 of 123 repositories (mean $\Delta = -0.016$). Of these, 20 showed decreased fan-out and 8 showed increased fan-out. The predominance of fan-out reduction is consistent with the frequent selection of coupling-reducing tactics (Decomposability and Reduced Coupling), though the magnitude is small. Notably, package count remained unchanged in all 123 repositories, indicating that the LLM modified intra-module structure without altering the top-level package organization.

**Docstring coverage.** Documentation coverage changed in 17 of 123 repositories (mean $\Delta = +0.004$), with 10 showing increased coverage and 7 showing decreased coverage. This marginal net improvement suggests that the LLM occasionally added docstrings during refactoring, but documentation improvement was not a primary effect of the applied tactics.

These supplementary metrics indicate that the LLM's interventions primarily affected code-level complexity (captured by MI) rather than high-level architectural structure (fan-out, packages). This reveals a key tension: the pipeline *selects* tactics at the architectural level but *implements* them through code-level changes—an observation consistent with the "Scale Gap" identified in prior work [9]. Bridging this gap between architectural intent and implementation scope remains an open challenge.

## 6 Threats to Validity

**Internal validity:** Each repository uses itself as a control, but MI changes could be influenced by factors beyond the tactic applied (e.g., code formatting changes). A single LLM model was used without comparison to alternatives or manual implementation.

**External validity:** Results are limited to Python repositories from GitHub. The Qwen3-coder-next model's behavior may not generalize to other LLMs, languages, or project types.

**Construct validity:** MI is a composite metric that may not capture all dimensions of maintainability. Architecture detection was performed by the LLM without human validation. The documented inconsistency among static analysis tools [12] further limits metric reliability.

**Conclusion validity:** While the Wilcoxon test shows statistical significance, the effect size is small ($r = 0.28$). The effective sample of improved repositories (22/162) is small, limiting the strength of conclusions.

## 7 Conclusion

This study investigated the potential of LLMs to automate architectural tactic implementation for software maintainability improvement. Regarding our research questions:

**RQ1:** The pipeline produced architecture classifications for all 162 repositories with high self-reported confidence (median 0.95, range 0.85–0.98). Tactic selection was coherent with detected architectures (e.g., 94% of script-based repositories received Decomposability), suggesting plausible reasoning. However, without independent ground-truth labels, detection accuracy cannot be formally validated—a key limitation.

**RQ2:** LLM-implemented tactics produced a statistically significant but small positive effect on MI (mean $\Delta$MI = +0.48, p = 0.001). 22 repositories showed measurable improvement, though 76.0% of completed cases remained unchanged and 25.3% failed.

**RQ3:** Script-based architectures showed the highest improvement potential, while modular monoliths exhibited both modest gains and all observed degradations. Reduced Coupling was the most effective tactic by improvement rate.

The high failure rate (25.3%) and prevalence of neutral results highlight limitations of current LLM pipelines for architecture-level modifications. Future work should prioritize: (1) integrating a TDD verification loop to ensure behavioral preservation; (2) multi-model comparison (GPT-4, Claude, Gemini) with a larger, architecturally diverse dataset; and (3) expert validation of architecture detection to establish ground-truth accuracy.

## Data Availability

The replication package, including pipeline code, experiment data, and analysis scripts, is available at https://github.com/ANONYMIZED.

## References


[1] 2023. ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://iso25000.com/en/iso-25000-standards/iso-25010. [Online; accessed 2025-12-04].

[2] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley.

[3] Justus Bogner, Stefan Wagner, and Alfred Zimmermann. 2019. Using architectural modifiability tactics to examine evolution qualities of Service- and Microservice-Based Systems. *Computer Science – Research and Development* 34 (2019), 187–237. doi:10.1007/s00450-019-00402-z

[4] Ali Changizi, Mohammad Dashti, Kian Dorrani, Tommaso Fulcini, Riccardo Coppola, and Flavio Giobergia. 2025. Enhancing Software Maintainability Through LLM-Assisted Code Refactoring. In *Generative AI for Effective Software Development*. Springer, 153–176.

[5] Marco Esposito, Xiaozhou Li, Sara Moreschini, Naveed Ahmad, and Valentina Lenarduzzi. 2025. Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions. *SSRN Preprint* (2025). Presented at ICSA 2025.

[6] Martin Fowler. 2018. *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.

[7] David Garlan and Mary Shaw. 1993. An introduction to software architecture. 1, 3 (1993).

[8] Maurice H. Halstead. 1977. *Elements of Software Science*. Elsevier.

[9] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. In *arXiv preprint arXiv:2511.04824*. arXiv:2511.04824

[10] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An Empirical Study of Refactoring Challenges and Benefits at Microsoft. *IEEE Transactions on Software Engineering* 40, 7 (2014), 633–649. doi:10.1109/TSE.2014.2318734

[11] Suntae Kim, Dae-Kyoo Kim, Lunjin Lu, and Sooyong Park. 2009. Quality-driven architecture development using architectural tactics. 82, 8 (2009), 1211–1231.

[12] Valentina Lenarduzzi, Fabiano Pecorelli, Nyyti Saarimaki, Savanna Lujan, and Fabio Palomba. 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. 198 (2023), 111575.

[13] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* 32, 1 (2025). doi:10.1007/s10515-025-00500-0

[14] Gastón Márquez, Hernán Astudillo, and Rick Kazman. 2022. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* 197 (2022), 111558. doi:10.1016/j.jss.2022.111558

[15] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. 2025. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* (2025), 112762. doi:10.1016/j.jss.2025.112762

[16] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837

[17] Paul Oman and Jack Hagemeister. 1992. Metrics for assessing a software system's maintainability. In *Conference on Software Maintenance*. IEEE, 337–344.

[18] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. 17, 4 (1992), 40–52. doi:10.1145/141874.141884

[19] Yonnel Chen Kuang Piao, Jean Carlos Paul, L. Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa, and Foutse Khomh. 2025. Refactoring with LLMs: Bridging Human Expertise and Machine Understanding. In *arXiv preprint arXiv:2510.03914*. arXiv:2510.03914

[20] Ana Paula Pucho, Anderson M. Ferreira, and Elder J. R. Cirilo. 2025. Refactoring Python Code with LLM-Based Multi-Agent Systems: An Empirical Study in ML Software Projects. In *Proceedings of the XXXIX Brazilian Symposium on Software Engineering (SBES)*. SBC.

[21] Zahed Rahmati and Mohammad Tanhaei. 2021. Ensuring software maintainability at software architecture level using architectural patterns. 2, 1 (2021), 81–102. doi:10.22060/ajmc.2021.19232.1044

[22] Seyyed Ehsan Shokri and Raffi Khatchadourian. 2024. IPSynth: Interprocedural Program Synthesis for Software Architecture Recovery and Architectural Tactics Detection. In *arXiv preprint arXiv:2403.10836*. arXiv:2403.10836

[23] Yisen Xu, Feng Lin, Jinqiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalis. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. In *arXiv preprint arXiv:2503.14340*. arXiv:2503.14340